PRD — Tablero de mensajes para Marcos

Oct 2, 2026 · @Nicolas

Contexto y objetivo

Marcos deja Incu y queremos regalarle un tablero donde cada persona del equipo le deje un mensaje, una foto o ambas cosas. El tablero se comparte por un link privado: quien tiene la URL puede verlo y escribir, sin crear cuenta.

Objetivos del producto:

- Que cualquier miembro del equipo pueda dejar su mensaje en menos de 2 minutos desde el celular.
- Que Marcos reciba un único link con todos los mensajes y fotos, que pueda conservar después de irse.
- Que el tablero sea reutilizable para futuras despedidas, cumpleaños o aniversarios (cada tablero con su propio ID).

Stack definido: Next.js (App Router) y Supabase (Postgres + Storage).

Alcance

El MVP cubre crear un tablero, compartirlo por URL, dejar mensajes con foto opcional y verlos en un muro. Lo demás queda para después.

Entra en el MVP	Queda fuera
Crear tablero con título, destinatario y foto de portada	Login de usuarios (SSO, Google, magic link)
Acceso por URL con ID no adivinable	Listado público de tableros
Mensaje de texto (hasta 1.000 caracteres) + nombre del autor	Reacciones, comentarios entre mensajes
Una foto opcional por mensaje (JPG, PNG, HEIC, WebP, hasta 10 MB)	Video, audio, GIFs animados
Muro tipo mosaico responsive	Exportar a PDF o libro impreso
Panel de administración por link secreto: ocultar o borrar mensajes, cerrar el tablero	Notificaciones por mail o Slack

Entra en el MVP	Queda fuera
Editar o borrar el propio mensaje desde el mismo dispositivo	Multi-idioma
Modo "revelación" opcional: el tablero se ve completo recién en una fecha	Plantillas visuales múltiples

Usuarios y flujos

Hay tres roles y ninguno necesita cuenta: el organizador, los participantes y el homenajeado. El rol lo define qué link tiene cada uno.

Rol	Qué link tiene	Qué puede hacer
Organizador	Link de administración (`/b/{boardId}/admin?key={adminKey}`)	Configurar el tablero, moderar mensajes, cerrar o reabrir, activar la revelación
Participante (equipo Incu)	Link público del tablero (`/b/{boardId}`)	Ver mensajes, publicar el suyo, editar o borrar el propio
Marcos	Mismo link público, o uno de solo lectura	Ver el muro completo y descargar fotos

Flujo del organizador:

1. Entra a la home y completa título, nombre del homenajeado y portada.
2. Recibe dos links: el público para compartir y el de administración para guardar.
3. Comparte el link público por Slack o WhatsApp con el equipo.
4. Antes de la entrega revisa el muro desde el panel y oculta lo que haga falta.
5. Cierra el tablero a nuevos mensajes y le envía el link a Marcos.

Flujo del participante:

1. Abre el link público y ve el muro con los mensajes existentes (o un placeholder si está en modo revelación).
2. Toca "Dejar un mensaje", escribe su nombre y el texto, y opcionalmente sube una foto.
3. Ve una previsualización y publica.
4. Su mensaje aparece en el muro; desde ese dispositivo puede editarlo o borrarlo mientras el tablero siga abierto.

Requisitos funcionales

ID	Requisito	Prioridad
RF-01	Crear un tablero con título, nombre del homenajeado, mensaje de bienvenida y foto de portada opcional	Must
RF-02	Al crear, generar un `boardId` público y un `adminKey` secreto, y mostrar ambos links con botón de copiar	Must
RF-03	Ver el muro en `/b/{boardId}`, ordenado por fecha de publicación (más antiguo primero)	Must
RF-04	Publicar un mensaje con nombre (obligatorio, hasta 60 caracteres), texto (hasta 1.000) y foto opcional; texto o foto deben estar presentes	Must
RF-05	Subir una foto por mensaje; el cliente la comprime a máximo 2.000 px de lado y convierte HEIC a JPEG antes de subir	Must
RF-06	El autor puede editar o borrar su mensaje desde el mismo navegador mediante un `editToken` guardado en localStorage	Should
RF-07	Panel de administración: listar mensajes, ocultar, restaurar y borrar	Must
RF-08	Cerrar el tablero a nuevos mensajes; el muro sigue visible	Must
RF-09	Modo revelación: hasta la fecha `revealAt`, los participantes ven solo cuántos mensajes hay y no su contenido	Could
RF-10	Ver una foto en grande (lightbox) y descargarla	Should
RF-11	Descargar todas las fotos en un ZIP desde el panel de administración	Could
RF-12	Vista de presentación a pantalla completa que rota mensajes, para la despedida presencial	Could
RF-13	Un `boardId` inexistente responde 404 sin revelar si el tablero existe	Must

Acceso por URL

La seguridad se basa en "capability URLs": conocer el ID es la autorización. Por eso el ID tiene que ser imposible de adivinar o enumerar, y nunca se usa el ID autoincremental de la base en una URL.

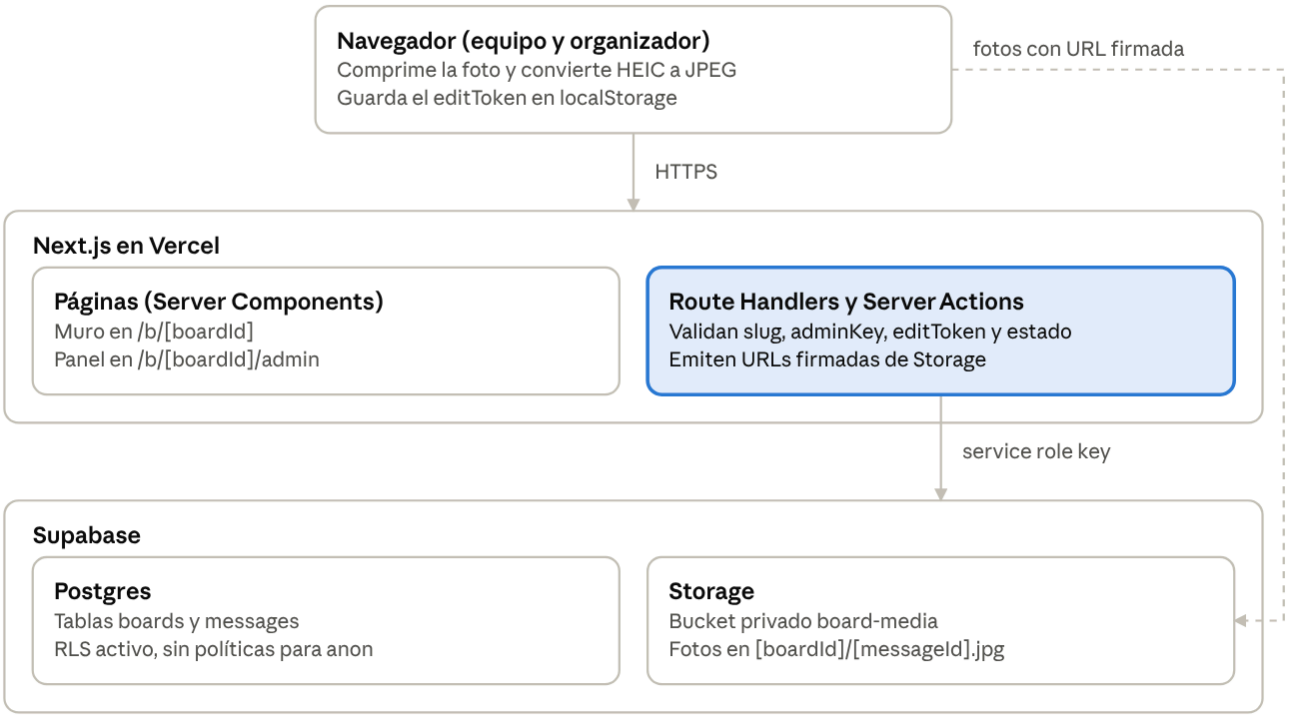
- **`boardId`**: 21 caracteres aleatorios con nanoid (alfabeto URL-safe, ~126 bits). Se guarda en una columna `slug` única, separada del `id` UUID interno.
- **`adminKey`**: 32 bytes aleatorios en base64url. En la base se guarda solo su hash SHA-256; el valor en claro se muestra una única vez al crear el tablero.
- **`editToken`** por mensaje: mismo esquema que el `adminKey`, hash en la base, valor en localStorage del autor.
- El cliente nunca habla directo con las tablas. Toda lectura y escritura pasa por Route Handlers o Server Actions de Next.js que usan la service role key de Supabase y validan slug, tokens y estado del tablero. RLS queda activado y sin políticas para `anon`, así la anon key no sirve para listar tableros.
- Las páginas de tablero llevan `noindex, nofollow` y `Referrer-Policy: no-referrer`, para que el link no se filtre en buscadores ni en el header Referer de las fotos.
- Rate limiting por IP en la creación de mensajes (por ejemplo 10 por minuto) y validación de tipo y tamaño de archivo del lado del servidor.
- Las fotos viven en un bucket privado; el servidor entrega URLs firmadas con vencimiento de 1 hora al renderizar el muro.

Riesgo aceptado: cualquiera que reciba el link reenviado puede escribir. Lo mitigamos con moderación desde el panel y con el cierre del tablero.

Arquitectura técnica

Next.js con App Router desplegado en Vercel, y Supabase como base de datos y almacenamiento de fotos. El navegador solo habla con Next.js; Next.js habla con Supabase con la service role key, que nunca llega al cliente.

El navegador nunca toca las tablas: todo pasa por el servidor



arquitectura · navegador, Next.js y Supabase

La única excepción al paso por el servidor son los bytes de las fotos: el servidor emite una URL firmada y el navegador sube o descarga directo de Storage. Así evitamos el límite de 4,5 MB por request de las funciones de Vercel.

Ruta	Tipo	Función
`/`	Página	Formulario para crear un tablero
`/b/[boardId]`	Página (Server Component)	Muro con mensajes y botón para dejar uno
`/b/[boardId]/admin?key=…`	Página	Panel de moderación; valida el hash del `adminKey`
`createBoard`	Server Action	Crea el tablero y devuelve los dos links
`requestUploadUrl`	Server Action	Valida el tablero abierto y devuelve una URL firmada de subida (`createSignedUploadUrl`)
`createMessage` / `updateMessage` / `deleteMessage`	Server Actions	Validan con zod, verifican `editToken` y escriben en Postgres

Ruta	Tipo	Función
`moderateMessage` / `setBoardStatus`	Server Actions	Requieren `adminKey` válido

Librerías: `@supabase/supabase-js`, `nanoid`, `zod`, `browser-image-compression` y `heic2any` en el cliente, y Tailwind para estilos. El mosaico se resuelve con CSS columns, sin librería extra.

Modelo de datos

Alcanzan dos tablas y un bucket. Los tokens se guardan hasheados y los mensajes ocultos se marcan, no se borran, para poder restaurarlos.

```
create table boards (
  id              uuid primary key default gen_random_uuid(),
  slug            text not null unique,          -- boardId de la URL (nanoid
21)
  title           text not null,
  recipient_name  text not null,
  welcome_message text,
  cover_path      text,                          -- ruta en Storage
  admin_key_hash  text not null,
  status          text not null default 'open' check (status in
('open','closed')),
  reveal_at       timestamptz,                   -- null = sin modo
revelación
  created_at      timestamptz not null default now()
);

create table messages (
  id              uuid primary key default gen_random_uuid(),
  board_id        uuid not null references boards(id) on delete cascade,
  author_name     text not null check (char_length(author_name) between 1 and
60),
  body            text check (char_length(body) <= 1000),
  photo_path      text,
  photo_width     int,
  photo_height    int,
  edit_token_hash text not null,
  is_hidden       boolean not null default false,
  created_at      timestamptz not null default now(),
  updated_at      timestamptz not null default now(),
  constraint has_content check (body is not null or photo_path is not null)
);
```

```
create index messages_board_created_idx on messages (board_id, created_at);

alter table boards   enable row level security;
alter table messages enable row level security;
-- Sin políticas para anon ni authenticated: solo el servidor (service role)
accede.
```

Storage: bucket privado `board-media`, con rutas `{boardId}/cover.jpg` y `{boardId}/{messageId}.jpg`. El ancho y alto de la foto se guardan para que el mosaico reserve el espacio y no salte al cargar.

Requisitos no funcionales

Aspecto	Requisito
Mobile	Diseño mobile-first; publicar un mensaje con foto desde un celular en 4G en menos de 10 segundos
Performance	Muro con 50 mensajes y fotos: LCP menor a 2,5 s; imágenes con `next/image`, lazy loading y tamaños responsive
Imágenes	Compresión en el cliente (máximo 2.000 px, calidad 0,8) y eliminación de metadatos EXIF, incluida la geolocalización
Accesibilidad	Contraste AA, navegación por teclado, texto alternativo con el nombre del autor en cada foto
Privacidad	Sin analytics de terceros en las páginas de tablero; política de borrado a pedido de Marcos o del organizador
Escala	Pensado para decenas de tableros y cientos de mensajes; el plan gratuito de Supabase alcanza (1 GB de Storage)
Disponibilidad	Deploy en Vercel; sin SLA formal
Moderación	Sin filtro automático en el MVP; moderación manual desde el panel

Criterios de aceptación y plan de entrega

El MVP (todos los Must) se estima en 3 a 4 días de un desarrollador; los Should y Could suman 2 días más.

Criterios de aceptación:

- [ ] Un tablero nuevo devuelve link público y link de administración, y el `adminKey` en claro no queda guardado en la base.
- [ ] Un `boardId` alterado en un carácter responde 404.
- [ ] La anon key de Supabase no puede leer ninguna fila de `boards` ni de `messages`.
- [ ] Desde un iPhone se publica un mensaje con una foto HEIC y se ve correctamente en el muro.
- [ ] La foto publicada no conserva metadatos de geolocalización.
- [ ] Un mensaje oculto desde el panel desaparece del muro y se puede restaurar.
- [ ] Con el tablero cerrado, el formulario no se muestra y la API rechaza nuevos mensajes.
- [ ] Las páginas de tablero incluyen `noindex` y `Referrer-Policy: no-referrer`.

Plan:

1. Día 1: setup de Next.js y Supabase, esquema, creación de tablero y links.
2. Día 2: muro, formulario de mensaje y subida de fotos.
3. Día 3: panel de administración, cierre del tablero, edición del propio mensaje.
4. Día 4: pulido mobile, pruebas de aceptación y deploy.
5. Después: revelación, lightbox, ZIP y modo presentación, si llegamos antes de la despedida.

Preguntas abiertas

- ¿Cuál es la fecha de la despedida de Marcos? Define si llegamos con los Could.
- ¿Marcos debe ver los mensajes a medida que llegan o es sorpresa hasta la entrega (modo revelación)?
- ¿Se permite que participen personas fuera de Incu, como clientes o ex compañeros?
- ¿Lo publicamos en un subdominio propio (por ejemplo `tablero.incu…`) o alcanza con el dominio de Vercel?
- ¿Lo pensamos solo para Marcos o lo dejamos listo como herramienta interna reutilizable desde el día uno?